Objective:

The goal of this lab is to help students understand and analyze basic time complexities: O(n), O (log n), O(n^2), O(n^3), and O (n log n). Students will implement algorithms that correspond to these time complexities, run them on data of varying sizes, and measure the execution time.

Problem Statement:

Create a program that reads data from files of different sizes (100, 1000, 5000, and 10000 entries) and executes simple algorithms representing different time complexities. Measure the time taken by each algorithm and store the results in a file.

Steps to follow:

1. Implement the algorithms in C++
2. Write a function to read data from the files into a dynamically allocated array. Use the data from files for running your algorithms.
3. Write the back result of all algorithms.
4. Compute the time of each algorithm using <chrono> library in C++
5. Record the time in milliseconds.
6. Save the time taken by each algorithm for each data size in a text file named “results.txt”

Pseudo codes

Following are the Pseudocodes of above algorithms:

1. **O(n): Find the Maximum Value in an Array**

```
Function findMax(array A, integer n):
  integer max = A[0]
  For i from 1 to n-1:
    If A[i] > max:
      max = A[i]
  Return max
```

2. **O(n^2): Selection Sort**

```
Function selectionSort(array A, integer n):
  For i from 0 to n-2:
    integer minIndex = i
    For j from i+1 to n-1:
      If A[j] < A[minIndex]:
        minIndex = j
    Swap A[i] with A[minIndex]
```

3. **O(n^3): Triple Nested Loop**

```
Function countTriplets(array A, integer n):
    integer count = 0
    For i from 0 to n-1:
        For j from i+1 to n-1:
            For k from j+1 to n-1:
                count = count + 1
    Return count
```

4. **O (n log n): Merge Sort**

```
Function mergeSort(array A, integer left, integer right):
    If left < right:
        integer mid = left + (right - left) / 2
        mergeSort(A, left, mid)
        mergeSort(A, mid + 1, right)
        merge(A, left, mid, right)

Function merge(array A, integer left, integer mid, integer right):
    integer n1 = mid - left + 1
    integer n2 = right - mid
    Create array L of size n1
    Create array R of size n2

    For i from 0 to n1-1:
        L[i] = A[left + i]
    For j from 0 to n2-1:
        R[j] = A[mid + 1 + j]

    integer i = 0
    integer j = 0
    integer k = left

    While i < n1 and j < n2:
        If L[i] <= R[j]:
            A[k] = L[i]
            i = i + 1
        Else:
            A[k] = R[j]
            j = j + 1
        k = k + 1
```

```
While i < n1:
    A[k] = L[i]
    i = i + 1
    k = k + 1

While j < n2:
    A[k] = R[j]
    j = j + 1
    k = k + 1
```

5. **O (log n): Binary Search**

```
Function binarySearch(array A, integer n, integer x):
    integer left = 0
    integer right = n - 1
    While left <= right:
        integer mid = left + (right - left) / 2
        If A[mid] == x:
            Return mid
        Else If A[mid] < x:
            left = mid + 1
        Else:
            right = mid - 1
    Return -1
```

```
#include<iostream>
#include <chrono>
using namespace std;
using namespace std::chrono;

int main() {
    auto start = high_resolution_clock::now();
    fun();
    auto stop = high_resolution_clock::now();
    auto duration = duration_cast<milliseconds>(stop - start);
    cout<< duration.count() << " ms" << endl;
}
```